```
# Movie Review Sentiment Classification with an ANN

## Objective
Predict whether a movie review is **positive (1)** or **negative (0)** using a
feedforward neural network built with TensorFlow/Keras. This is a binary
classification problem.

## Dataset
- **File:** `movie_reviews.csv`
- **Size:** 200 rows, 2 columns, no missing values
- **Columns:** `review_text` (the review) and `sentiment` (positive/negative)
- **Note:** The dataset contains only 8 unique short reviews (e.g. "I loved the
  movie!", "Worst plot ever."), each repeated 25 times. It is a small
  proof-of-concept dataset, not real-world review data.

## Methodology
1. Loaded the data and checked its shape, data types, and missing values
2. Encoded both `review_text` and `sentiment` with `LabelEncoder`
3. Split the data 80/20 into training and test sets (`random_state=42`)
4. Scaled the features with `StandardScaler`
5. Built a Sequential ANN: Dense(32, ReLU) → Dense(16, ReLU) → Dense(1, Sigmoid)
   (Adam optimizer, binary cross-entropy loss)
6. Trained for up to 500 epochs with a 20% validation split and `EarlyStopping`
7. Evaluated with accuracy, loss, and a classification report

## Results
- Test accuracy: **87.5%**, test loss: **0.293**
- Class 0 (negative): precision 0.76, recall 1.00
- Class 1 (positive): precision 1.00, recall 0.79

## Limitations and Future Work
- Reviews were label-encoded into single integers, so the model sees review IDs,
  not the meaning of the words. Real text models need TF-IDF, word embeddings,
  or an LSTM/transformer.
- With only 8 unique sentences, the same reviews appear in both training and test
  sets, so the accuracy does not reflect performance on unseen reviews.
- Next steps: use a real review dataset and proper text vectorization, and
  evaluate on reviews the model has never seen.

## Tech Stack
Python, Pandas, Seaborn, Matplotlib, Scikit-learn, TensorFlow/Keras
```

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("movie_reviews.csv")
df.head()
```

	review_text	sentiment
0	I loved the movie!	positive
1	It was terrible!	negative
2	Fantastic acting!	positive
3	Worst plot ever.	negative
4	A masterpiece of cinema.	positive

```
df.shape
```

```
(200, 2)
```

```
df["review_text"].value_counts()
```

review_text	count
I loved the movie!	25
It was terrible!	25
Fantastic acting!	25
Worst plot ever.	25
A masterpiece of cinema.	25
Too long and boring.	25
Brilliant visuals!	25
Mediocre at best.	25

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   review_text  200 non-null    object
 1   sentiment    200 non-null    object
dtypes: object(2)
memory usage: 3.3+ KB
```

```
df.isnull().sum()
```

	0
review_text	0
sentiment	0

dtype: int64

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["sentiment"] = le.fit_transform(df["sentiment"])
df["review_text"] = le.fit_transform(df["review_text"])
```

```
df.head()
```

	review_text	sentiment
0	3	1
1	4	0
2	2	1
3	7	0
4	0	1

```
df.columns
```

```
Index(['review_text', 'sentiment'], dtype='object')
```

```
# Setting the target and Feature Variables
```

```
x = df[['review_text']]
y = df['sentiment']
```

```
from sklearn.model_selection import train_test_split
```

```
xtrain, xtest, ytrain, ytest = train_test_split(x, y, test_size = 0.2, random_state = 42)
xtrain.shape, xtest.shape, ytrain.shape, ytest.shape
```

```
((160, 1), (40, 1), (160,), (40,))
```

```
from sklearn.preprocessing import StandardScaler
```

```
SS = StandardScaler()
xtrain = SS.fit_transform(xtrain)
xtest = SS.fit_transform(xtest)
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()

model.add(Dense(32, activation = "relu", input_shape = (xtrain.shape[1],)))
model.add(Dense(16, activation = "relu"))
model.add(Dense(1, activation = "sigmoid"))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Compiling the model
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)

# Training the model
history = model.fit(
    xtrain, ytrain,
    batch_size=32,
    epochs=500,
    validation_split=0.2,
    callbacks=[early_stopping]
)
```

```
Epoch 1/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 2s 96ms/step - accuracy: 0.5703 - loss: 0.6671 - val_accuracy: 0.7500 - val_loss: 0.6617
Epoch 2/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9453 - loss: 0.6429 - val_accuracy: 1.0000 - val_loss: 0.6387
Epoch 3/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.6192 - val_accuracy: 1.0000 - val_loss: 0.6161
Epoch 4/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.5968 - val_accuracy: 1.0000 - val_loss: 0.5937
Epoch 5/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.5742 - val_accuracy: 1.0000 - val_loss: 0.5720
Epoch 6/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.5521 - val_accuracy: 1.0000 - val_loss: 0.5508
Epoch 7/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.5310 - val_accuracy: 1.0000 - val_loss: 0.5301
Epoch 8/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.5104 - val_accuracy: 1.0000 - val_loss: 0.5099
Epoch 9/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.4902 - val_accuracy: 1.0000 - val_loss: 0.4908
Epoch 10/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.4706 - val_accuracy: 1.0000 - val_loss: 0.4715
Epoch 11/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.4511 - val_accuracy: 1.0000 - val_loss: 0.4520
Epoch 12/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.4312 - val_accuracy: 1.0000 - val_loss: 0.4327
Epoch 13/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.4120 - val_accuracy: 1.0000 - val_loss: 0.4135
Epoch 14/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3927 - val_accuracy: 1.0000 - val_loss: 0.3946
Epoch 15/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3739 - val_accuracy: 1.0000 - val_loss: 0.3758
Epoch 16/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3553 - val_accuracy: 1.0000 - val_loss: 0.3574
Epoch 17/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3373 - val_accuracy: 1.0000 - val_loss: 0.3394
Epoch 18/500
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3197 - val_accuracy: 1.0000 - val_loss: 0.3222
Epoch 19/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.3028 - val_accuracy: 1.0000 - val_loss: 0.3055
Epoch 20/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2864 - val_accuracy: 1.0000 - val_loss: 0.2895
Epoch 21/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2710 - val_accuracy: 1.0000 - val_loss: 0.2743
Epoch 22/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 24ms/step - accuracy: 1.0000 - loss: 0.2562 - val_accuracy: 1.0000 - val_loss: 0.2598
Epoch 23/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 25ms/step - accuracy: 1.0000 - loss: 0.2420 - val_accuracy: 1.0000 - val_loss: 0.2460
Epoch 24/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 22ms/step - accuracy: 1.0000 - loss: 0.2288 - val_accuracy: 1.0000 - val_loss: 0.2329
Epoch 25/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 1.0000 - loss: 0.2161 - val_accuracy: 1.0000 - val_loss: 0.2204
Epoch 26/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.2040 - val_accuracy: 1.0000 - val_loss: 0.2085
Epoch 27/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.1925 - val_accuracy: 1.0000 - val_loss: 0.1973
Epoch 28/500
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 1.0000 - loss: 0.1818 - val_accuracy: 1.0000 - val_loss: 0.1868
Epoch 29/500
```

```
# Evaluating the model
loss, accuracy = model.evaluate(xtest, ytest)
print("Accuracy", accuracy)
print("Loss", loss)
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - accuracy: 0.8750 - loss: 0.2934
Accuracy 0.875
Loss 0.29343467950820923
```

```
# Making predictions
y_pred = model.predict(xtest)
y_pred[0:5]
```

```
2/2 ━━━━━━━━━━━━━━━━━━━━ 0s 128ms/step
array([[1.2656748e-06],
       [1.2656748e-06],
       [9.9999332e-01],
       [9.9999332e-01],
       [9.5640399e-02]], dtype=float32)
```

```
# Converting from Probability
y_pred = (y_pred > 0.5).astype(int)
y_pred[0:5]
```

```
array([[0],
       [0],
       [1],
       [1],
       [0]])
```

```
y_compare = pd.DataFrame(ytest)
y_compare['Predicted'] = y_pred
y_compare = y_compare.reset_index()
y_compare.drop('index', axis = 1)
```

	sentiment	Predicted
0	0	0
1	0	0
2	1	1
3	1	1
4	1	0
5	0	0
6	0	0
7	1	1
8	1	1
9	0	0
10	1	1
11	1	1
12	0	0
13	1	1
14	1	1
15	0	0
16	1	0
17	1	0

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
cr = classification_report(ytest, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support

           0       0.76      1.00      0.86        16
           1       1.00      0.79      0.88        24

    accuracy                           0.88        40
   macro avg       0.88      0.90      0.87        40
weighted avg       0.90      0.88      0.88        40
```

29	1	1
30	0	0
31	1	1
32	0	0
33	0	0
34	1	1
35	1	0
36	0	0